

exp-fips203-mlkem-pke-encrypt-k3

FIPS 203 Alg.14 ML-KEM-768 PKE Encrypt 实现方案

ascendc 工程 (ML-KEM-768 W4a / E14)

2026-07-26

摘要

本文档描述 `examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-encrypt-k3/` 在 Atlas A2 / Ascend910B4 上以 AscendC 实现 FIPS 203 Algorithm 14, 即 ML-KEM-768 ($k=3$) PKE Encrypt。

参数锁定: 几何、I/O 与 launch 全部锁定到 `docs/specs/fips203-mlkem768-parameter-card.md` §3.2 D14。**行为基线:** 活跃探针 `ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203-alg14-pke-encrypt-device-k3/`。**生产 I/O:** input/ 为 `ek_pke.bin` (1184B)、`m.bin` (32B)、`coins.bin` (32B) 与静态 LUT; output/ 仅 `c.bin` (1088B)。**计划 AI Core 数:** 2 launch; Launch 1 为 AIV_ONLY blockDim=2, Launch 2 为 MIX blockDim=1 (1 AIC + 2 AIV)。**禁止:** 创建 `examples/stable/ml-kem/ml-kem-768/`; 从任何 `frozen/` 目录复制、改写或移植源码; 用 k4 的 `ek=1568`、`c=1568`、 $\hat{A}[16]$ 、`re[9]` 或 INTT batch 6/8 替代 k3 几何。

目录

1	工程边界	2
1.1	允许的代码来源与依赖	2
1.2	禁止项	2
2	数学语义 (Alg.14, $k=3$, $n=256$, $q=3329$)	2
2.1	输入输出	2
2.2	阶段公式	3
3	AscendC 实现规划	3
3.1	Launch 与 AI Core 规模	3
3.2	GM 几何	3
3.3	同步与流水	4
4	AscendC API 表	4
4.1	评估过但未采用	5
5	数学步骤覆盖率	5

目录	2
6 验证计划	6
6.1 默认验收	6
6.2 可选对照	6
7 产出物	6

1 工程边界

1.1 允许的代码来源与依赖

来源	用途
ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203- alg14-pke-encrypt-device-k3/ examples/incubating/ml-kem/ml-kem-1024/exp-fips203-mlkem-pke-encrypt-k4/ library/shared/ CANN / tikicpulib / CAModel	活跃 D14 k3 行为基线；可复制后自包含适配到本 exp，最终不得保留对探针目录的编译期 include/import 活跃 k4 incubating 结构参考；仅 retarget 到 k3 锁定几何与字节长 SHAKE/Keccak、公共设备侧头文件、统一整数压缩辅助等共享库 编译、CPU 孪生与 SIM 运行环境

1.2 禁止项

- 禁止从 ascendc-tests/frozen/ 或 examples/frozen/ 拿源码、customspec 或路线。
- 禁止将 $\hat{A}$ 、 re 、 $\hat{r}$ 、 u 、 v 、 $\hat{t}$ 等中间态作为默认 I/O 落盘；debug dump 只能显式开关启用。
- 禁止把 k4 的 $\text{du}=11/\text{dv}=5$ 、 $c_1=1408/c_2=160$ 、 $\text{re}[9]$ 、INTT batch 6/8 或零垫 4/8 当作默认。
- 禁止新建或写入 stable-768；本目录只作为 incubating 预研实现。

2 数学语义 (Alg.14, $k = 3$, $n = 256$, $q = 3329$)

2.1 输入输出

张量 / 文件	契约
input/ek_pke.bin	1184B；前 1152B 为 $\hat{t}$ 的 ByteEncode_{12} ，末 32B 为 ρ
input/m.bin	32B 明文消息；尾段按 ByteDecode_1 后 Decompress_1 注入 v
input/coins.bin	32B Alg.14 随机性；用于 PRF 生成 $r\ e_1\ e_2$
input/lut_even_stacked.bin input/lut_odd_stacked.bin	NOT INTT Stage2 LUT；由本目录 golden 脚本生成；不是用户可见交付输入
output/c.bin	1088B, $c = c_1\ c_2$ ，其中 $c_1 = 960\text{B}$ ($d_u = 10$, 3 poly), $c_2 = 128\text{B}$ ($d_v = 4$, 1 poly)

2.2 阶段公式

1. 行 2: 从 `ek_pke[0:1152]` 解码 $\hat{t} \in R_q^3$, 此步骤在 compute launch 内完成, Host 不预解码落盘。
2. 行 3-7: 取 $\rho = \text{ek_pke}[1152:1184]$, 对 $(j, p) \in \{0, 1, 2\}^2$ 生成 $\hat{A}[j, p]$, GM 形状锁定为 `[9,256] int32`。
3. 行 8-15: 用 $\eta_1 = \eta_2 = 2$ 从 `coins` 采样 $r \|_{e_1 \| e_2}$ 共 7 个 poly; GM 形状锁定为 `[7,256]`。
4. 行 16-18: 计算 $\hat{r} = \text{NTT}(r)$, 再得 $\hat{u}_j = \sum_p \hat{A}[j, p] \circ \hat{r}_p$ 与 $\hat{tr} = \sum_p \hat{t}_p \circ \hat{r}_p$ 。
5. 行 19-21: 对 $\hat{u}_0, \hat{u}_1, \hat{u}_2, \hat{tr}$ 执行真 **INTT batch=4**; 得到 u_0, u_1, u_2, v 后分别加 e_1 与 $e_2 + \mu(m)$ 。
6. 行 22: $c_1 = \text{ByteEncode}_{10}(\text{Compress}_{10}(u))$, $c_2 = \text{ByteEncode}_4(\text{Compress}_4(v))$, 输出 $c = c_1 \| c_2$ 。

3 AscendC 实现规划

3.1 Launch 与 AI Core 规模

Launch	核类型 / blockDim	数据划分与理由
1 prep	AIV_ONLY, blockDim=2	$\hat{A}$ 共 9 poly 按 5+4 分给两个 AIV; $r \ _{e_1 \ e_2}$ 共 7 poly 由 block0 生成并按真实 7 poly 写 GM。选择 2 AIV 是参数卡 §3.2 D14 锁定值, 继承 D14 探针已验同步与 UB 分配。
2 compute+tail	MIX, blockDim=1	1 AIC + 2 AIV; $\text{NTT}(r)$ 只处理真实 3 poly; 内积输出 4 个待 INTT poly; INTT batch4 的 AIV 映射为连续 2+2。选择单 MIX AI Core 是 D14 锁定生产档, 避免跨 AI Core GM 交换。

3.2 GM 几何

对象	锁定几何
$\hat{A}$	<code>[9,256] int32</code> ; 9 个真实 poly, 禁止 pad 到 16
<code>re</code>	<code>[7,256] int32</code> ; <code>r[0..2] e1[0..2] e2[0]</code>
$\hat{r}$ NTT	<code>polyvec3</code> ; 复用 B5/NTT 真实 poly 几何, 不引入假 poly
INTT batch4	<code>S0 [8,256] int8</code> , <code>Stage2 mat_c [32,128] int32</code> ; Cube 的 M 对齐垫只是硬件对齐, 不表示假 poly
Tail pack	u 三 poly 用 $d_u = 10$ pack 为 960B; v 一 poly 用 $d_v = 4$ pack 为 128B

3.3 同步与流水

Prep 内部使用 UB/GM 分段搬运与必要的 PipeBarrier；两次 launch 之间由 Host 顺序保证 GM 可见性。Compute 内部遵守 D14/B5/B6 已验模式：AIV 写入平面 Stage1/Stage3 GM 后以 CrossCore flag 与 AIC 同步；AIC 写完 mat_c 后 AIV 等待再做 Stage3 merge、内积与 tail pack。CPU 仅作为逻辑孪生；SIM 是同步与搬运验收门槛。

4 AscendC API 表

本实现不引入 E14 独有的新 AscendC API；下表 API 均已有索引记录，使用约束按 library /documents/CANN-AscendC算子开发接口参考-查阅索引.md 执行。

API 名	分类 / 文档	Atlas A2	本用例 dtype	备注
GetBlockIdx / GetSubBlockIdx	系统变量 / 资源	√	标量索引	区分 AIV 分片与 MIX 子核；禁止把 CPU 打印误认为多核性能
GlobalTensor / LocalTensor	基础结构	√	uint8/int8/int32	GM/UB 张量契约必须与本 spec 几何一致
DataCopy	Memory 数据搬运	√	uint8/int8/int32	连续块搬运；长度与地址遵守 32B 对齐约束
PipeBarrier	Pipe 同步	√	—	MTE/Vector/Cube 阶段交界显式同步
CrossCoreSetFlag / CrossCoreWaitFlag	MIX 同步	√	—	AIV/AIC 通过 GM 交接时使用；CPU 过不能删
Mmad / Fixpipe	矩阵计算	√	int8*int8->int32	Stage2 MMAD；INTT batch4 的逻辑 M=8，硬件 M 对齐垫到 16
ShiftRight	矢量算术	√	int32	limb 拆分、Barrett 压缩与 ByteEncode 辅助右移
Muls / Add / Sub	矢量算术	√	int32/int16	limb、模加减、压缩/编码辅助

Cast	类型转换	√ (受限)	int32/int16/half/uint8	遵守 A2 链约束; 不得假设存在 int32->int8 单跳
Gather	矢量搬运/选择	√	int32	仅允许在 Alg.11 / ByteEncode 等非 NTT S1-S3 段按既有路径使用; NTT 三段内禁用
GetValue / SetValue	LocalTensor 标量访问	√	uint8/int32	仅用于 pack 尾部等标量缺口; 不得把 Compares bit 包当逐 lane 布尔

4.1 评估过但未采用

API / 路线	不采用原因
Compares/GatherMask compact 链	Alg.14 E14 不需要新增 reject compact; 沿用 D14 已验 pack 路径, 不新增比较掩码风险
高阶 Matmul<>	本路线继承 D14 手写 MMAD + 平面 mat_c 契约; 高阶 Matmul 输出布局与 MIX 交接不作为本轮变量
Gather 用于 NTT S1-S3	参数卡与 ML-KEM NTT 定稿禁止; NTT 三段保持平面 mat_c + 连续 poly 批处理

5 数学步骤覆盖率

数学步骤	实现方式 / API	覆盖	缺口说明
取 ρ 、采样 $\hat{A}$ [9]	AIV_ONLY + XOF/rej + DataCopy	100% 设备	5+4 分片; 无零垫
采样 $r\ e_1\ e_2$ [7]	AIV_ONLY + PRF/CBD	100% 设备	真实 7 poly; 不借 k4 9 poly
NTT(r) 与内积	MIX AIV/AIC + Mmad	100% 设备	NTT 内无 Gather
INTT batch4	MIX AIV/AIC + Stage3 merge	100% 设备	真 polyvec4, AIV 连续 2+2

Compress/ByteEncode $d = 10/4$	统一整数压缩 + pack	部分向量	pack 尾部标量缺 口沿用 D14/k4 已验实现
-----------------------------------	---------------	------	----------------------------------

6 验证计划

6.1 默认验收

```
cd examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-encrypt-k3
bash run.sh -r cpu -v Ascend910B4
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

期望: c.bin 为 1088B; 与 golden 对拍 [cmp] c max=0。SIM 通过后记录 Total tick 到 qa/active_sim_regress_summary.md, 并检查用例根目录无 stray core*.dump、profile_*log*.toml。

6.2 可选对照

后续可补 PKE round-trip 或 liboqs encrypt/decrypt 夹具, 但 E14 本轮门禁为 CPU + SIM_DIRECT=1 sim。liboqs 仅作为黑盒 I/O oracle; 禁止把 liboqs 源码实现同构移植进 AscendC。

7 产出物

- exp-fips203-mlkem-pke-encrypt-k3-实现方案-customspec.tex
- exp-fips203-mlkem-pke-encrypt-k3-实现方案-customspec.pdf
- 后续实现文件须与本 customspec 同目录自包含, 且自研 Python/C/AscendC 写详细中文注释。